

HipKittens x IRIS Meeting 5

Subha

July 28, 2026

Prefill/Decode Setups

- Prefill: TP=8, DP=1, EP enabled, GRAPH mode PIECEWISE
 - Batch Size: 4096
- Decode: TP=1, DP=8, EP=8, GRAPH mode FULL_DECODE_ONLY
 - Batch Size: 64, 128

Results

End-to-End Tested Results

REGIME	BEST CUSTOM PATH	LATENCY		PAIRED RATIO (OURS / PROD.)		VERDICT
		PRODUCTION	OURS	P50	P95	
Decode, B64	k0d_mega + k0d_combine	548.17 μ s	536.81 μ s	0.9781	1.0313	parity; $\sim$ 2.2% median gain
Prefill, B4096 (<i>shipped prod. capacity</i>)	pf3_pd	—	—	0.9634	0.9824	$\sim$ 3.7% median win
Prefill, B4096 (<i>equal capacity</i>)	pf3_pd	8.4145 ms	8.3007 ms	0.9840	0.9999	parity; $\sim$ 1.6% median gain

All measurements warm; values summarize five independent processes.

Ratios are paired and aggregated separately from the displayed median-run p50 latencies; values below 1.0 favor our path.

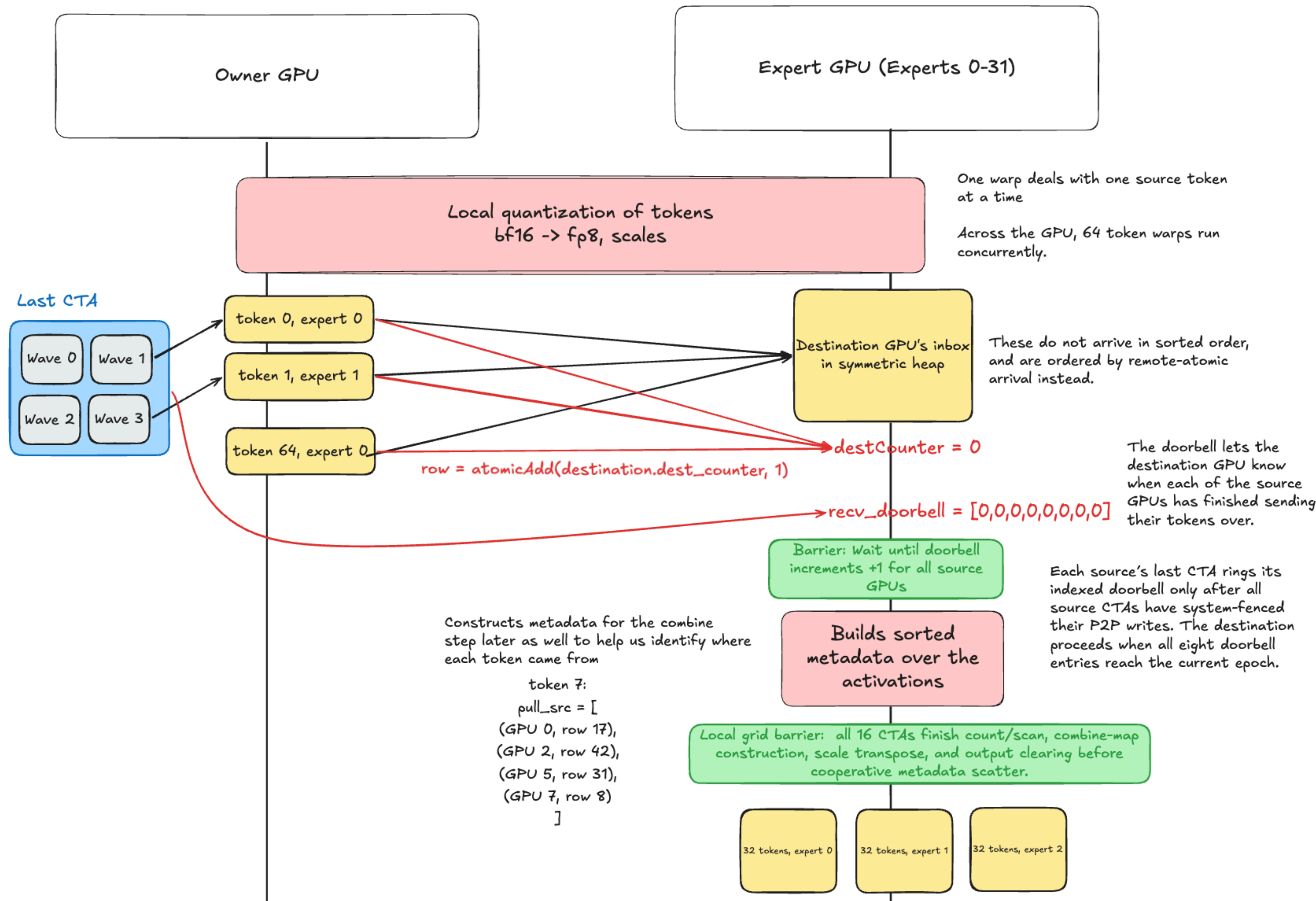
Shipped production capacity is MAXTOK = 32768 versus candidate MAXTOK = 4096; equal capacity uses 4096/4096.

- Shipped production capacity refers to the maximum workspace size configured in advance. The 32K production configuration pays additional capacity related overhead
- These are tested end-to-end on graph mode

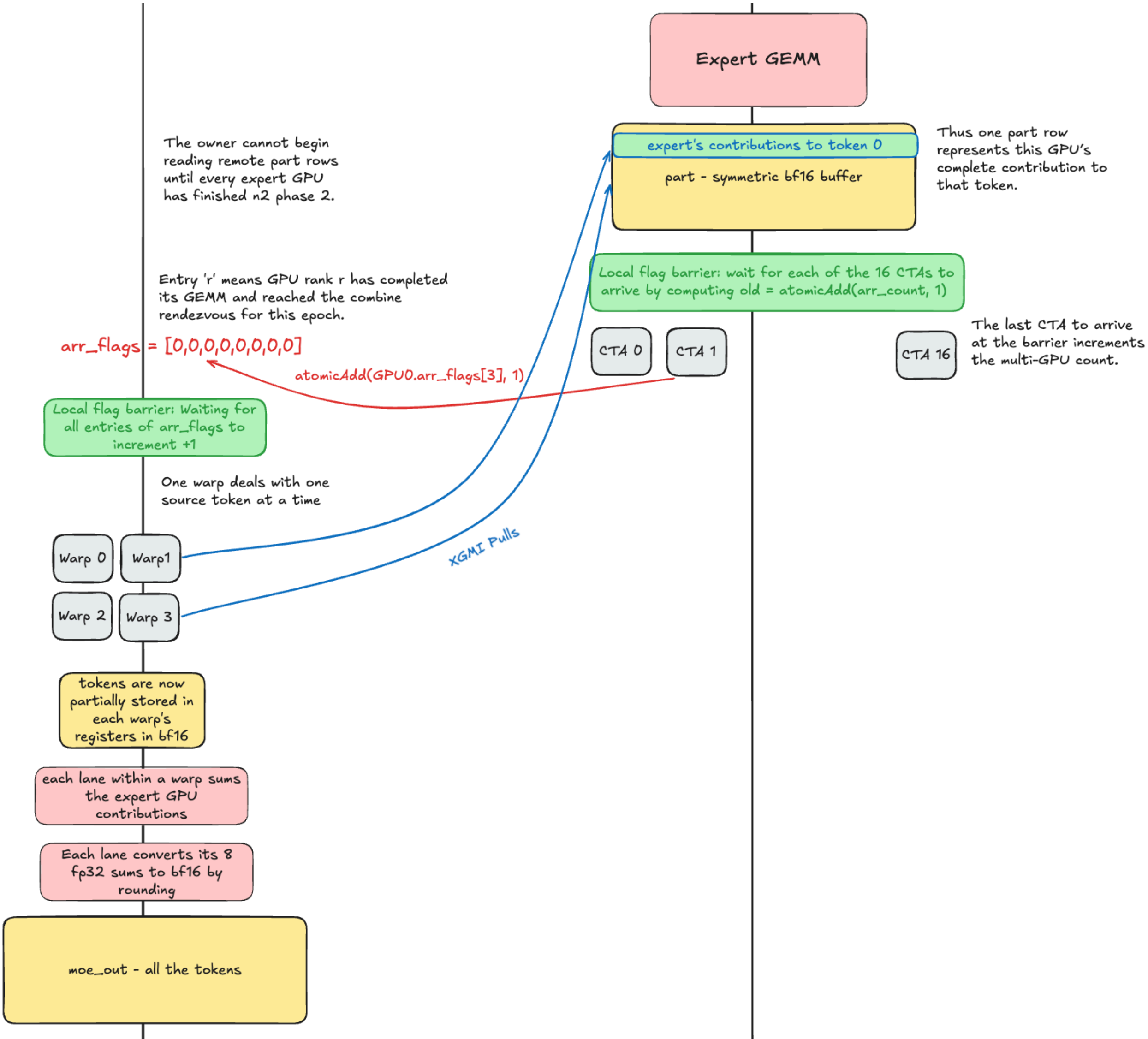
Decode Kernels

Decode Kernels

Kernel Data Flow Diagram



Kernel Data Flow Diagram



Profiling Analysis

CAPTURED PHASE	LATENCY		CUSTOM – PRODUCTION		INTERPRETATION
	PRODUCTION	CUSTOM	$\Delta \mu s$	$\Delta \%$	
Quantization	—	11.81	—	—	Separate custom graph node
Ingress	28.55	33.27	+4.72	+16.53	Small remaining transport deficit
GEMM phase	422.85	377.88	−44.97	−10.63	Custom graph-level advantage
Combine phase	51.11	26.27	−24.84	−48.60	Custom direct-pull combine is faster
Instrumented region	520.27	511.33	−8.94	−1.72	Small critical-path advantage

All latencies in μs . Δ is custom minus production, so negative favors the custom path: -10.63% means custom is 10.63% faster.

Quantization has no production counterpart; it is folded into the production `fmoe` node, so no delta is available.

Phases are measured independently and need not sum to the instrumented region.

- This table follows the slowest GPU rank through the complete MoE region
- While `k0d_mega` fuses push and destination processing into one launch, the destination still has to wait for all of the sources to finish which causes this synchronization tail
- In our pre-GEMM region, one wave handles one token and loops serially over its destination GPUs

Pre-GEMM Profiling Analysis

COMPONENT	LATENCY	SHARE	WHAT LIMITS IT
M1 token push sweep	≈ 26.7	$\approx 80\%$	One wavefront processes ≈ 3.4 destinations serially: ≈ 10 peer-memory instructions per destination, or ≈ 34 per token
Fence + completion flags + wait + destination metadata	≈ 6.6	$\approx 20\%$	Remaining budget includes publication, cross-GPU waiting, sorting, scans, scale transpose, and output clearing
Complete k0d_mega ingress	33.27	100%	Captured steady-state graph measurement

All latencies in μs . Shares are of the complete ingress measurement.

- This is just for the region before the expert GEMM. The token push dominates this region
- However, the XGMI fabric is under used since we achieve only 60 GB/s instead of 240-250 in our earlier probe. This is because one wavefront/token processes 3.4 destinations serially
- Also due to limited Concurrency: 64 wavefronts vs 8192 scheduled wavefronts
- Planned change (potentially): Use 2-4 wavefronts per token and split its destination list, perhaps could save around 10-17 microseconds

Doorbell Mechanism Breakdown

Current Implementation (Inefficient)

Table: doorbell mechanism

Doorbell Implementation	Rendezvous Breakdown			Relative Result
	Fan-out p50	Wait p50	Total p50	
Serial atomics, current	5.73	0.80	6.89	Baseline
Eight-lane atomics	0.70	0.72	1.68	Best; same protocol
Serial epoch stores	6.29	0.97	7.55	Store type alone does not help
Eight-lane epoch stores	1.50	0.35	2.28	Better than serial, slower than lane atomics

All latencies in μ s. Bold marks the current implementation and the best candidate.

```
one lane:
  notify GPU 0
  notify GPU 1
  notify GPU 2
  ...
  notify GPU 7
```

Better Version

```
lane 0 → notify GPU 0
lane 1 → notify GPU 1
...
lane 7 → notify GPU 7
```

- Fan-out p50 refers to the time for one source GPU to publish “I’m done” to all other 8 GPUs
- Wait p50 refers to the time the destination spends waiting after it’s doorbell check
- Total is the entire synchronization sequence, including publication, waiting, etc.
- Epoch store writes the desired epoch value directly instead of atomically incrementing it, however serial atomics vs serial stores didn’t really help, so the main problem is that one wavefront issues several remote instructions *serially* without enough outstanding work

Design Decisions

- Push vs Pull
 - On the 8x MI350x node, push4 is measured at 240-250 GB/s compared to pull's 177-190 GB/s
 - The reason that the pull based implementation requires a barrier is because the pull gather reads REMOTE quantized rows, so the remote quantization must be complete
- ROCm/DeepEP has an interesting implementation as well, however they include additional machinery for large M and use RDMA
- HIP Stream overlap is not currently possible since that would only exist across region boundaries (e.g. next layer's quant/push vs this layer's GEMM tail) and would require other changes, could potentially look into this next.
- Our current implementation uses 5 kernel launches (quantization, k0d_mega, n2_phase 1 GEMM, n2 phase 2 GEMM, k0d_combine)

Decode GEMM Results

Implementation	Kernel Breakdown		Total
	GEMM Core	Epilogue	
Production AITER	396.44	29.88	426.33
Our n2	415.47	3.65	419.11

All times in μ s. Totals as reported; components may not sum exactly due to rounding.

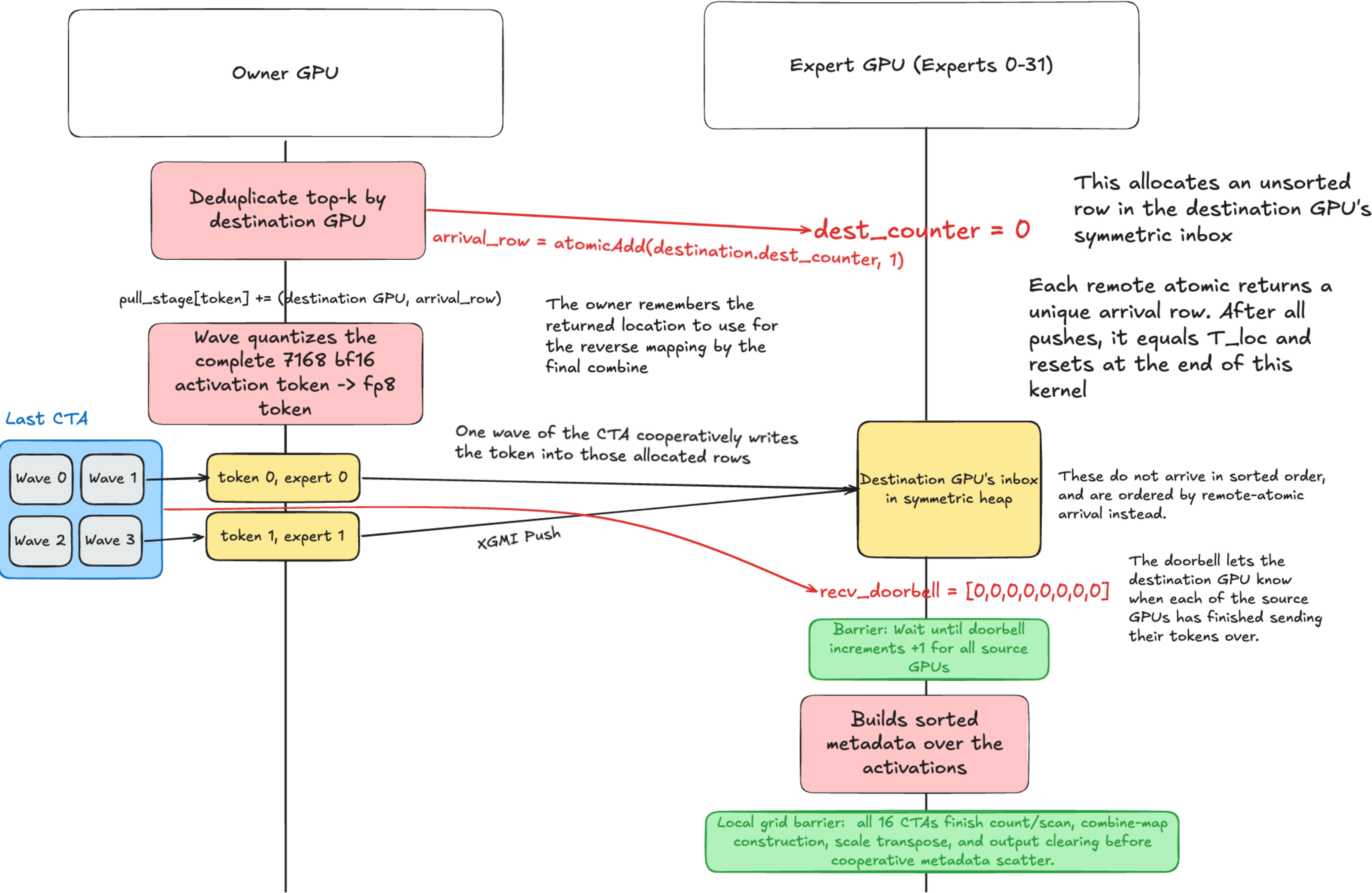
- The epilogue refers to the portion where we are preparing and writing the matrix multiply results into the per-GPU part buffer that the combine kernel consumes
 - GEMM epilogue (inside n2): scale, bf16 conversion, and write partial outputs to part[row]
- Production AITER uses a split-K design, so all 8 pieces must publish partial results to the same output. This creates 8 partial writes x 22.9 MB = 183.5 MB of atomic output traffic
- Our n2 CTA has already completed the full K reduction in registers, and it only needs to publish the result once so the traffic is only 22.9 MB

Planned Changes

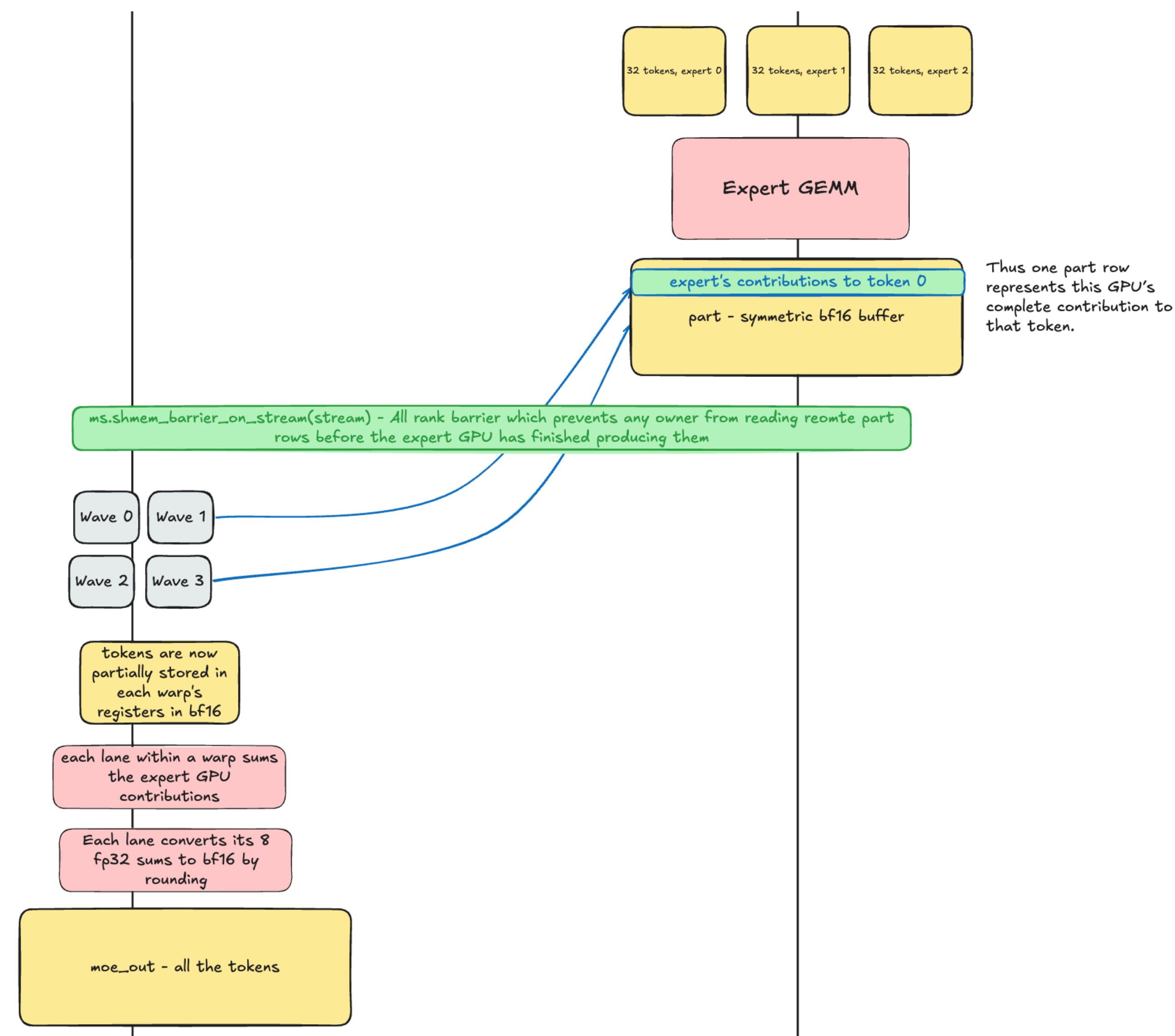
- Use 2-4 waves per token and split the destination list. This should shorten the 34-instruction sequential remote chain
- Replace serial doorbell loops with 8-lane atomics
-

Prefill Kernels

Kernel Data Flow Diagram



Kernel Data Flow Diagram



Design Decisions